

PRÁCTICA 1

Programación con MIPS64

Conceptos desarrollados:

- Tamaño de los operandos.
- Formato de la memoria.
- Representación de operandos.
- Implementación de vectores.
- Primeros programas.

Primera parte

Vamos a hacer un programa muy sencillo para familiarizarnos con la forma de programar a bajo nivel, en nuestro caso, en el lenguaje ensamblador de Mips64. En el primer apartado se muestra el esqueleto de un programa que suma el contenido de las variables A y B dejando el resultado en C.

Podemos apreciar que tanto A como B tienen un tamaño de 64 bits y que sus valores iniciales son 4 y F respectivamente, este último expresado en hexadecimal. Por el contrario, la variable C se ha definido como un conjunto de 8 bytes sin valor inicial.

También podemos ver que el código objeto se va a generar a partir de la dirección 12 (C hexadecimal) de la caché de código debido a la directiva "org". Al no haber puesto ninguna directiva similar en la zona de datos, estos se generarán a partir de la dirección cero de su respectiva caché.

- 1) Completar el programa para que realice correctamente su función. Este esqueleto se proporciona en el fichero Suma.s.

```
.data

A:    .word  4
B:    .word  0xF
C:    .space 8

.text
.org  0xC                                ld R1, A(R0);
                                           ld R2, B(R0);
                                           dadd R3, R1, R2;
                                           sd R3, C(R0);
                                           halt
```

- 2) Anotar las direcciones de todos los bytes ocupados por las variables A, B y C.

Variable	Direcciones ocupadas
A	0000-0007
B	0008-000F
C	0010-0017

- 3) Anotar el código objeto y las direcciones de los bytes ocupados por la instrucción `halt`.

Instrucción	Código objeto	Direcciones ocupadas
halt	04000000	001C-001F

Segunda parte

Ahora A y B serán dos vectores de 8 elementos, de tamaño byte, cada uno de los cuales podrá almacenar un número entero con valor entre 0 y 15. El programa debe realizar la suma de ambos vectores dejando el resultado en otro vector C de características similares.

- 1) Completar el esqueleto de programa para que realice su cometido. Se suministra en Vectores.s.

```

.data
.org 32      ; 32(10) = 20(16)

A:   .byte 0,9,11,3,12,5,14,7
B:   .byte 8,1,10,2,13,4,6,15
C:   .byte 0,0,0,0,0,0,0,0

.text
      . . . . .
      . . . . .
      . . . . .
fin:  halt

```

`daddi R10,R0,0`
`daddi R5,R0,0`
for: `slti R5,R10,8;`
`beqz R5, fin`
`lb R1, A(R10)`
`lb R2, B(R10)`
`dadd R3,R1,R2`
`sb R3,C(R10)`
`daddi R10,R10,1;`



- 2) Anotar la dirección ocupada por los elementos indicados en la tabla.

Elemento	B [0]	A [3]	B [7]	C [5]
Dirección	0028	0023	002F	0035

- 3) Modificar el programa para que el tamaño de los elementos de los vectores sea de 32 bits y comprobar que funciona correctamente.

- 4) Anotar las nuevas direcciones ocupadas por los elementos indicados en la tabla. Tener en cuenta que ahora cada elemento ocupa 4 bytes.

Elemento	B [0]	A [3]	B [7]	C [5]
Direcciones	0040-0043	002C-002F	005C-005F	0074-0077

3) .data

.org 32 ;

A: .word32 0,9,11,3,12,5,14,7

B: .word32 8,1,10,2,13,4,6,15

C: .word32 0,0,0,0,0,0,0,0

.text

daddi R10,R0,0

daddi R5,R0,0

for: slti R5,R10,32

beqz R5, fin

lb R1, A(R10)

lb R2, B(R10)

dadd R3,R1,R2

sb R3,C(R10)

daddi R10,R10,4

j for

fin: halt